МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
<БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ>

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность АС-68

Выполнил
В А Веремейчук
студент группы АС-68

Проверил
А. А. Крощенко,
доцент кафедры ИИТ,

«_> _ _____2026 г.

Брест 2026

Цель: На практике сравнить работу нескольких алгоритмов классификации, таких как метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научиться подбирать гиперпараметры моделей и оценивать их влияние на результат.

Вариант 3

- Wine Quality
- Классифицировать вино на "хорошее" (оценка >= 7) и "обычное" (оценка < 7)
- **Задания:**
    1. Загрузите данные и создайте бинарную целевую переменную;
    2. Стандартизируйте признаки и разделите выборку;
    3. Обучите модели k-NN, Decision Tree и SVM;
    4. Сравните F1-score для каждой модели, так как классы могут быть несбалансированы;
    5. Определите, какой алгоритм показал наилучший баланс между точностью и полнотой.

Выполнение**:**

Код программы

```
from pathlib import Path

import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import classification_report

def main():
    folder = Path(__file__).resolve().parent
    df = pd.read_csv(folder / "winequality-white.csv", sep=";")

    X = df.drop(columns="quality")
    y = (df["quality"] >= 7).astype(int)

    print("\nКлассы: 0 — обычное вино, 1 — хорошее вино")
    print(y.value_counts().sort_index().rename("Количество").to_string())

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    print("\nОбучающая выборка:", len(X_train))
    print("Тестовая выборка:", len(X_test))

    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

    knn_search = GridSearchCV(
        Pipeline([
            ("scaler", StandardScaler()),
            ("knn", KNeighborsClassifier())
        ]),
        {"knn__n_neighbors": list(range(1, 26))},
        scoring="f1",
        cv=cv,
        n_jobs=-1
    )

    knn_search.fit(X_train, y_train)
    best_k = knn_search.best_params_["knn__n_neighbors"]
```

```
    print("\nЛучшее k по кросс-валидации:", best_k)
    print(f"F1 на кросс-валидации: {knn_search.best_score_:.4f}")

    k_results = pd.DataFrame({
        "k": list(range(1, 26)),
        "mean_f1": knn_search.cv_results_["mean_test_score"],
        "std_f1": knn_search.cv_results_["std_test_score"]
    })

    print("\nВлияние количества соседей:")
    print(k_results.to_string(index=False, float_format=lambda x: f"{x:.4f}"))
    k_results.to_csv(folder / "knn_results.csv", index=False)

    models = {
        "k-NN": knn_search.best_estimator_,
        "Decision Tree": DecisionTreeClassifier(random_state=42),
        "SVM": Pipeline([
            ("scaler", StandardScaler()),
            ("svm", SVC(kernel="rbf", C=1.0, gamma="scale"))
        ])
    }

    results = []

    for name, model in models.items():
        if name != "k-NN":
            model.fit(X_train, y_train)

        prediction = model.predict(X_test)

        results.append({
            "Модель": name,
            "Accuracy": accuracy_score(y_test, prediction),
            "Precision": precision_score(y_test, prediction, zero_division=0),
            "Recall": recall_score(y_test, prediction, zero_division=0),
            "F1-score": f1_score(y_test, prediction, zero_division=0)
        })

        print(f"\n{name}:")
        print(classification_report(
            y_test,
            prediction,
            target_names=["Обычное вино", "Хорошее вино"],
            digits=4,
            zero_division=0
        ))

    results = pd.DataFrame(results).sort_values("F1-score", ascending=False)

    print("\nСравнение моделей на тестовой выборке:")
    print(results.to_string(index=False, float_format=lambda x: f"{x:.4f}"))

    results.to_csv(
        folder / "model_results.csv",
        index=False,
        encoding="utf-8-sig"
    )

    print(f"\nЛучший алгоритм: {results.iloc[0]['Модель']}")

if __name__ == "__main__":
    main()
```

Результат:

Классы: 0 — обычное вино, 1 — хорошее вино
quality
0 3838
1 1060

Обучающая выборка: 3918
Тестовая выборка: 980

Лучшее k по кросс-валидации: 1
F1 на кросс-валидации: 0.6206

Влияние количества соседей:

k	mean_f1	std_f1
1	0.6206	0.0192
2	0.5078	0.0276
3	0.5740	0.0235
4	0.5086	0.0136
5	0.5514	0.0121
6	0.5051	0.0168
7	0.5373	0.0129
8	0.4839	0.0073
9	0.5271	0.0106
10	0.4801	0.0173
11	0.5132	0.0114
12	0.4720	0.0176
13	0.4952	0.0091
14	0.4624	0.0217
15	0.4964	0.0214
16	0.4609	0.0252
17	0.4846	0.0223
18	0.4523	0.0352
19	0.4791	0.0286
20	0.4486	0.0296
21	0.4663	0.0273
22	0.4396	0.0281
23	0.4633	0.0282
24	0.4446	0.0338
25	0.4545	0.0358

k-NN:

	precision	recall	f1-score	support
Обычное вино	0.9217	0.9193	0.9205	768
Хорошее вино	0.7103	0.7170	0.7136	212
accuracy			0.8755	980
macro avg	0.8160	0.8181	0.8170	980
weighted avg	0.8759	0.8755	0.8757	980

Decision Tree:

	precision	recall	f1-score	support
Обычное вино	0.9070	0.8763	0.8914	768
Хорошее вино	0.6008	0.6745	0.6356	212
accuracy			0.8327	980
macro avg	0.7539	0.7754	0.7635	980
weighted avg	0.8408	0.8327	0.8360	980

SVM:

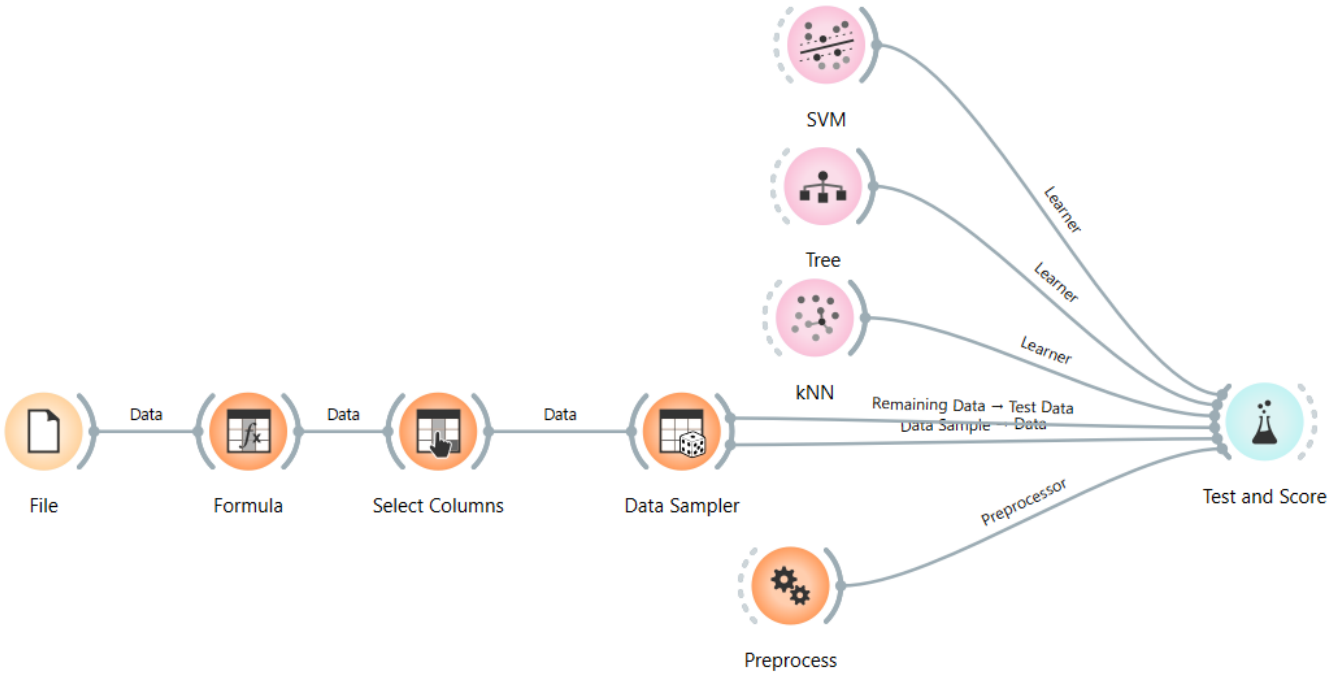
	precision	recall	f1-score	support
Обычное вино	0.8375	0.9661	0.8972	768
Хорошее вино	0.7234	0.3208	0.4444	212
accuracy			0.8265	980
macro avg	0.7804	0.6435	0.6708	980
weighted avg	0.8128	0.8265	0.7993	980

Сравнение моделей на тестовой выборке:

Модель	Accuracy	Precision	Recall	F1-score
k-NN	0.8755	0.7103	0.7170	0.7136
Decision Tree	0.8327	0.6008	0.6745	0.6356
SVM	0.8265	0.7234	0.3208	0.4444

Лучший алгоритм: k-NN

Результаты в Orange:



Model	AUC	CA	F1	Prec	Recall	MCC
SVM	0.367	0.698	0.383	0.343	0.434	0.189
Tree	0.732	0.820	0.585	0.585	0.585	0.470
kNN	0.777	0.846	0.648	0.641	0.656	0.549

Вывод: В ходе лабораторной работы выполнена классификация вина с помощью k-NN, дерева решений и SVM. Наилучший результат показал k-NN при k = 1: F1-score составил 0,7136 в Python и 0,648 в Orange. Этот метод обеспечил лучший баланс точности и полноты среди рассмотренных моделей